

EPR-07-TEAM/Restaurant/Dokumentation:

Am Anfang des Programmcodes importieren wir **csv**, damit man später eine Speisekarte aus einer Datei einlesen kann. In der nächsten Zeile importieren wir **datetime**, um das Datum und die Uhrzeit auf die Rechnung zu speichern und anzuzeigen.

Zuerst sehen wir drei verschiedene Kategorien, **class Food**, **class Order**, und **class MenuReader**.

Der class Food der ersten Funktion ist wie eine Speisekarte. Dafür gibt es in der ersten Funktion die Methode **__init__**, die jedem Essen oder Getränk im Programm eine/n **Namen, Art, Kategorie und Preis** gibt.

Die darauffolgende Funktion **show_item** ist wie ein Kellner, der dann sagt, was ein Essen kostet und zu welcher Kategorie es gehört. Es zeigt nur Informationen des Essens auf dem Bildschirm an.

In der **class Order** Kategorie geht es darum Bestellungen für einen Tisch zu verwalten. Hier ist wiederum die **__init__**-Methode der Startpunkt, damit alles für eine Bestellung vorbereitet ist. Durch **self.table_number** kann die Nummer des Tisches, der bestellt hat, gespeichert werden. Dann erstellt **self.items = []** auf der darauffolgenden Zeile eine leere Liste, wo dann alle bestellten Speisen und Getränke gespeichert werden. Außerdem steht **self.special_wish = []** eine leere Liste für Sonderwünsche wie z.B: „extra Käse“ dar. Danach folgt der **self.not.paid = True**. Hier bedeutet True, dass die Rechnung noch nicht bezahlt wurde, sobald aber ein Kunde bezahlt, ändern sich dies zu **False**. Der **self.total_price = 0** startet im Prinzip bei null Euro, wenn erstmal noch nichts bestellt wurde.

Durch die nächste Funktion **def add_foods** wird Essen zur Bestellung hinzugefügt. Und wenn der Kunde einen extra Wunsch hat (z.B. Extrakäse), wird dann der Preis durch **(+=)** im Programmcodes um 1 Euro erhöht. Sobald der Wunsch die Wörter „extra“ oder „more“ enthält, wird 1 Euro daraufgesetzt. Durch **print** wird eine Nachricht angezeigt, die dann folgendermaßen lauten könnte: *„Pizza mit extra Käse (1EUR added to the price!)*. Darüberhinaus wird zunächst **else** verwendet. Denn im nächsten *print* wird deutlich gemacht, dass eine Bestellung nicht mehr hinzugefügt werden kann, wenn die Rechnung abgeschlossen bezahlt wurde. In diesem Fall vermittelt dann *print* folgende Nachricht: *“Can not add more item, the bill already successfully paid!”*.

Danach folgt die Funktion **def remove_food**, um Essen aus der Bestellung zu entfernen, wenn es noch nicht bezahlt wurde. Dafür überprüft die Funktion zuerst, ob die Rechnung bereits bezahlt wurde. Wenn nicht, dann darf der Kunde was aus der Bestellung entfernen. Damit das geschieht, geht die Funktion die Liste mit den bestellten Speisen durch, um den Namen des Gerichts zu entfernen. Dabei wird dann durch **(-=)** im Programmcodes, der Preis vom Gesamtpreis abgezogen. Erwähnenswert hierbei ist, dass bei Sonderwünschen 1 Euro zusätzlich abgezogen wird. Wenn aber die Rechnung schon bezahlt ist, wird durch *print* die Nachricht weitergegeben, dass das bestellte Essen nicht mehr entfernt werden kann.

Es kann auch vorkommen, dass das Essen, welches man entfernen möchte, nicht gefunden wird. Hierfür erscheint durch **print** dann z.B.: „*Pizza is not in the order! Check again, maybe you made a typing mistake.*”

Danach wird die Funktion **def mark_paid** erstellt, wo der Wert **self.not_paid** in diesem Moment von *True auf False* geändert wird, sodass die Rechnung als bezahlt markiert wird und deshalb auch keine Änderungen mehr möglich sind. Es wird dann wiederum durch *print* vermittelt, für welche Tischnummer die Rechnung bezahlt wurde. Eine weitere Nachricht zeigt an, dass man nichts mehr ändern kann.

Zunächst erstellt man die Funktion **def invoice** , was die Aufgabe hat die Rechnung auf dem Bildschirm (Bestellübersicht) anzuzeigen. Hierfür wird die aktuelle Uhrzeit ermittelt und formatiert. Es wird durch „*/n” die Rechnungsüberschrift angezeigt. Danach wird durch die **for-schleife**, wird der Name und der Preis von der Bestellung sowie Sonderwunsch falls vorhanden angezeigt. Am Ende zeigt die Funktion durch *print* den Gesamtpreis an. Diese Funktion erstellt eine Art Kassenzettel.

Die andere Funktion **def save_invoice** speichert dann die Rechnung in einer Textdatei. Sie ist wie ein Kassenzettel-Drucker. Hierfür wird die Textdatei mit den Namen „rechnung.txt” geöffnet. Zuerst wird die Rechnung hinzugefügt und dann die bestellten Speisen mit den jeweiligen Preisen. Der Sonderwunsch wird hier auch betrachtet. Schließlich wird dann der Gesamtpreis gespeichert. Das ist hilfreich, damit der Kunde/ die Kundin sich später **die** Bestellung gedruckt anschauen kann.

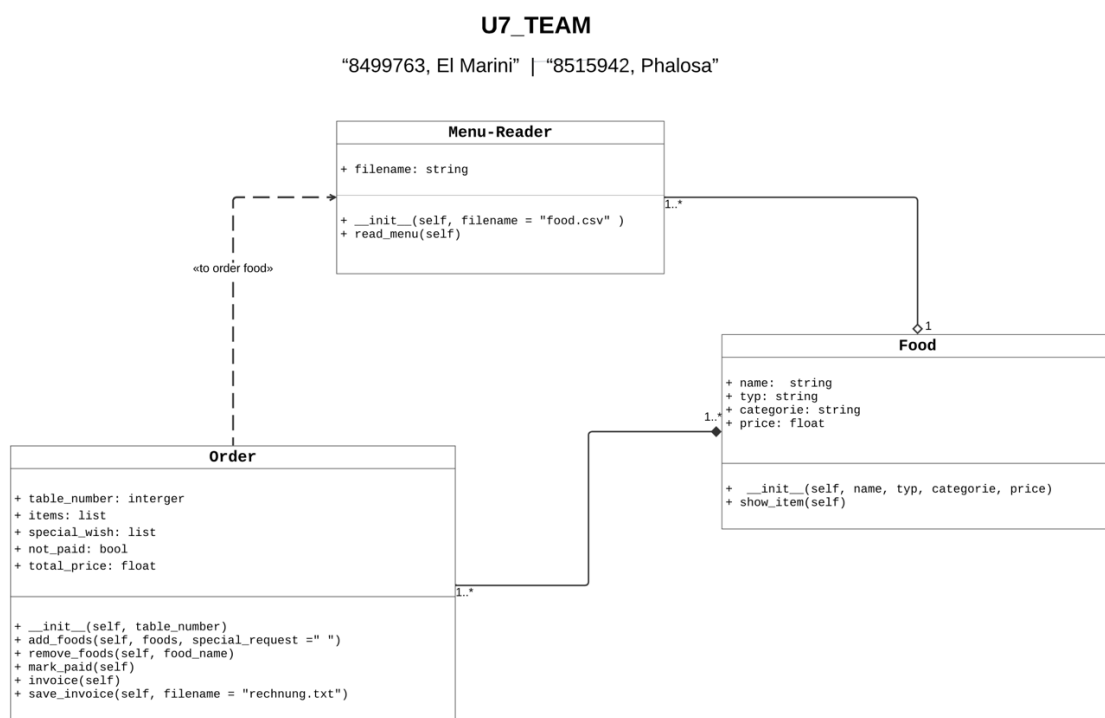
In Class Menu Reader gibt es die Funktion **def read_menu**. Die Funktion **def read_menu** liest eine Speisekarte aus der Datei **food.csv** im Lesemodus „r”, um die Gerichte zu lesen. Jede Zeile enthält nämlich Informationen zum Gericht mit ihrem Typ, Kategorie und Preis. Beinhaltet der Preis Punkte oder Kommas, wird dieser richtig formatiert und in eine Zahl umgewandelt, sodass man später damit kalkulieren kann. Die ganzen Gerichte werden in eine Liste für später gespeichert und durch **return** zurückgegeben.

In **def main** sind die Testfälle vorhanden. Es ist wichtig, um sicherzustellen, dass keine Fehler passieren. Man möchte nämlich falsche Eingaben verhindern, und dass Rechnungen korrekt erstellt werden etc.

Zuerst wird getestet, ob der Nutzer eine gültige Tischnummer eingegeben hat. Falls die Eingabe ungültig ist, tritt eine Fehlermeldung auf: „*Invalid number! Please enter a valid number for your table.*” Dann fragt das Programm erneut nach der Eingabe. Zunächst wird getestet, welche Menüoptionen von 1 bis 4 der Nutzer wählt. Durch *print* werden diese vier verschiedenen Optionen angezeigt. Diese **while-Schleife** fragt also immer wieder nach deinen Wünschen, solange du nicht fertig bist. Zunächst wird die erste Option überprüft, ob ein gültiges Gericht aus dem Menü gewählt wurde. Falls dies nicht der Fall ist, wird dies wiederum als Fehlermeldung angezeigt. Gibt der Nutzer aber „**exit**” ein, dann beendet das Programm die Bestellung (hier *break* verwendet). Die zweite Option wäre, ob ein Gericht aus der Bestellung entfernt werden kann. Aber man muss hier auch berücksichtigen, was passiert, wenn der Nutzer etwas Falsches als Gericht eingibt. Denn hier soll eine Fehlermeldung erscheinen. Bei der

dritten Option wird getestet, ob es funktioniert, die Rechnung als bezahlt zu markieren. Man verwendet hier *break*, um an dieser Stelle die Bestellung nicht fortzusetzen. Falls die Rechnung aber bezahlt wurde, wird dies als Benachrichtigung in der Konsole angezeigt. Schließlich können durch *if __name__ == "__main__"* können die Testfälle ordentlich ausgeführt werden.

UML Klassendiagramm



➔ Es gibt die Testfälle unten.

“8499763, El Marini ; 8515942, Phalosa”

Die Testfälle

[illegible]

“8499763, El Marini ; 8515942, Phalosa”

[illegible]

Die Ergebnis in Rechnung.txt

